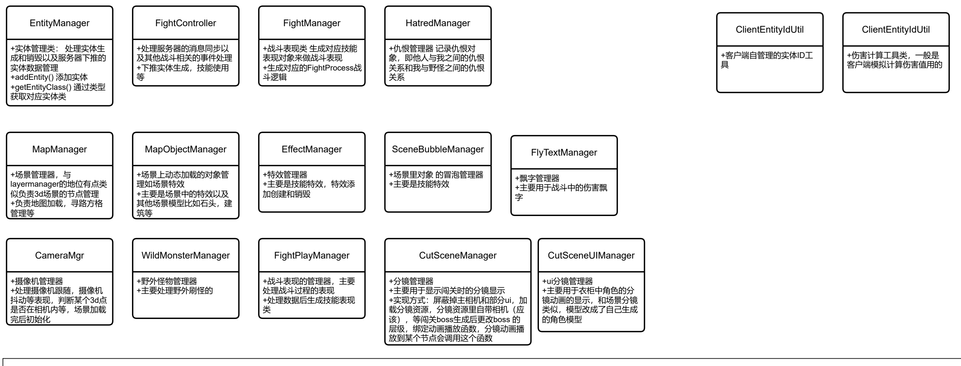
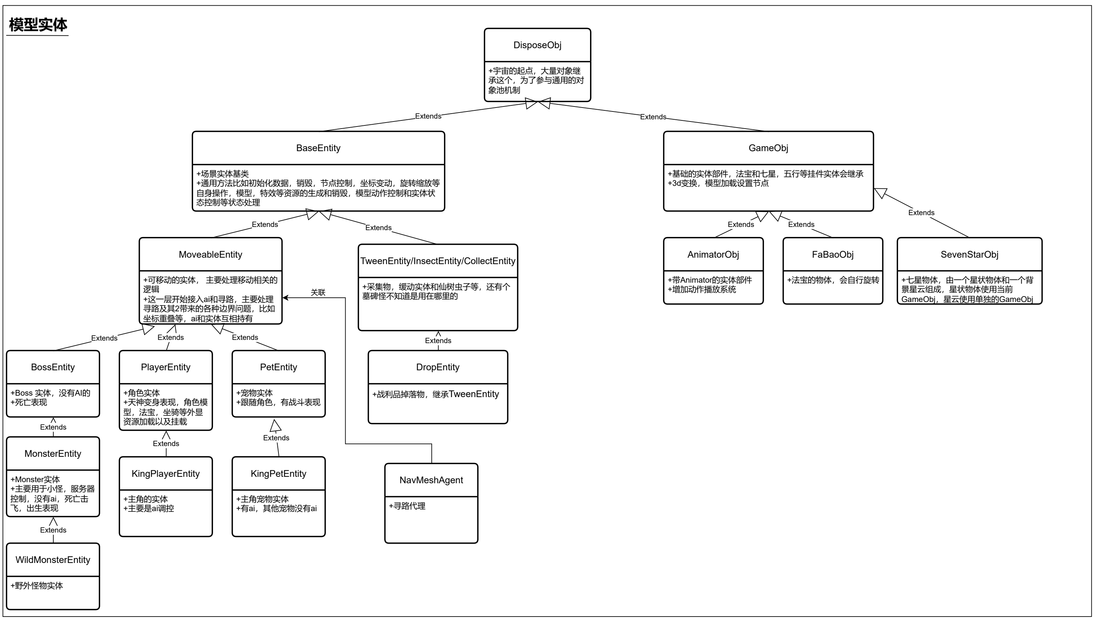
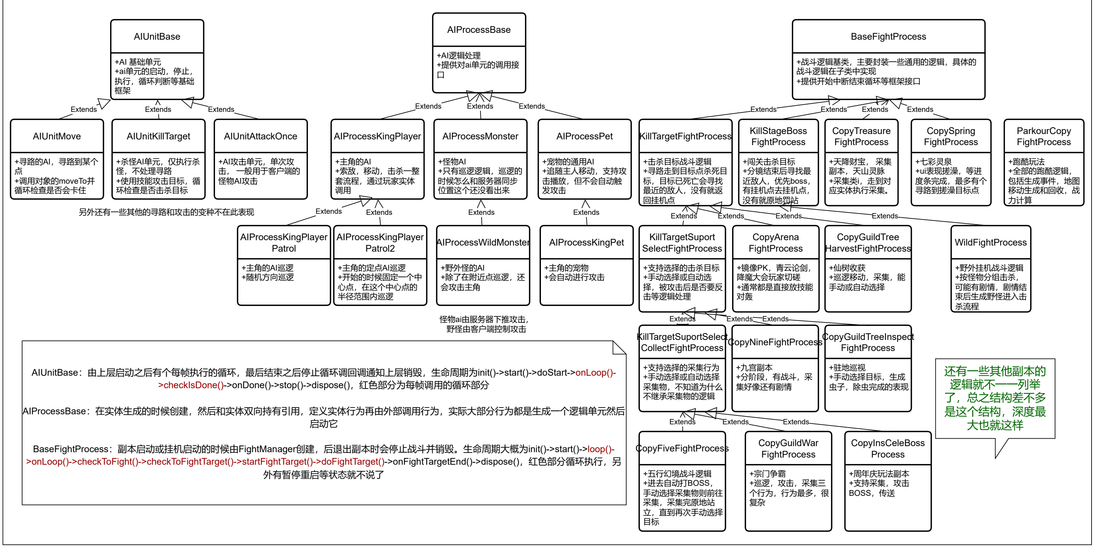


模型实体



ai逻辑

继承AIUnitBase一套的类是基础ai单元，负责单个行动ai比如 （杀死目标行为），继承AIProcessBase一套的类是多个ai单元组成的实体行为ai逻辑
最后继承BaseFightProcess一套的类是控制多个实体及其AIProcessBase来完成副本或者一场战斗表现的类
大概就是类似unit->process->fightprocess这样的三层结构



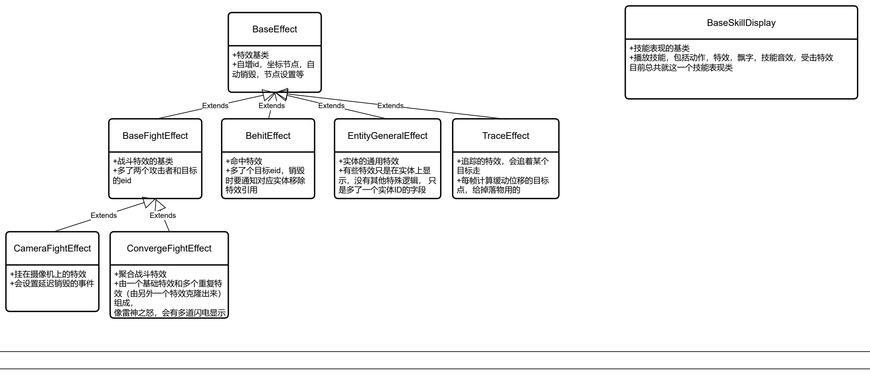
AIUnitBase：由上层启动之后有个每帧执行的循环，最后结束之后停止循环调回调通知上层销毁，生命周期为init()->start()->doStart->onLoop()->checkIsDone()->onDone()->stop()->dispose()，红色部分为每帧调用的循环部分

AIProcessBase：在实体生成的时候创建，然后和实体双向持有引用，定义实体行为再由外部调用行为，实际大部分行为都是生成一个逻辑单元然后启动它

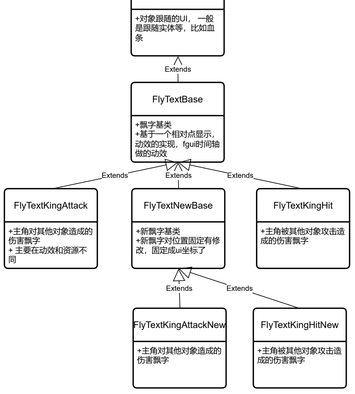
BaseFightProcess：副本启动或挂机启动的时候由FightManager创建，后退出副本时会停止战斗并销毁。生命周期大概为init()->start()->loop()->onLoop()->checkToFight()->checkToFightTarget()->startFightTarget()->doFightTarget()->onFightTargetEnd()->dispose()，红色部分循环执行，另外有暂停重启等状态就不说了

还有一些其他副本的逻辑就不一一列举了，总之结构差不多是这个结构，深度最大也就这样

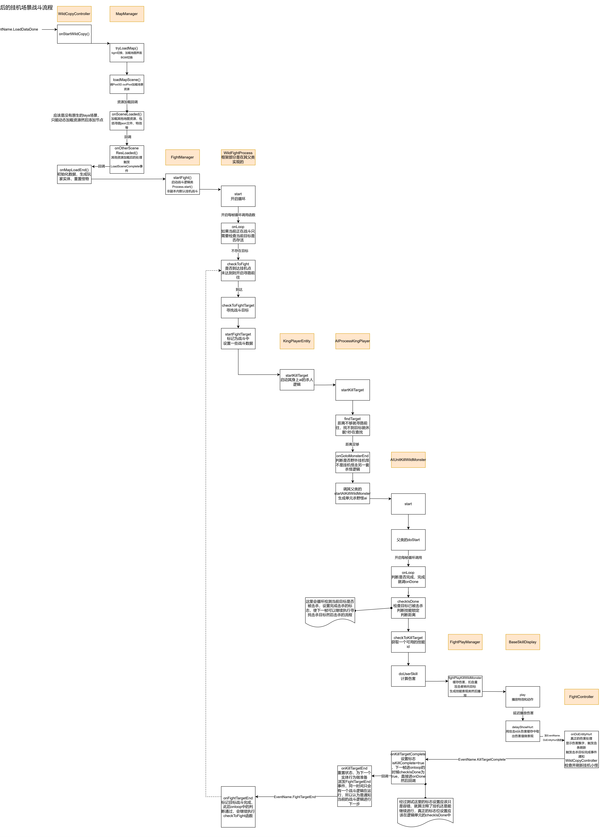
特效

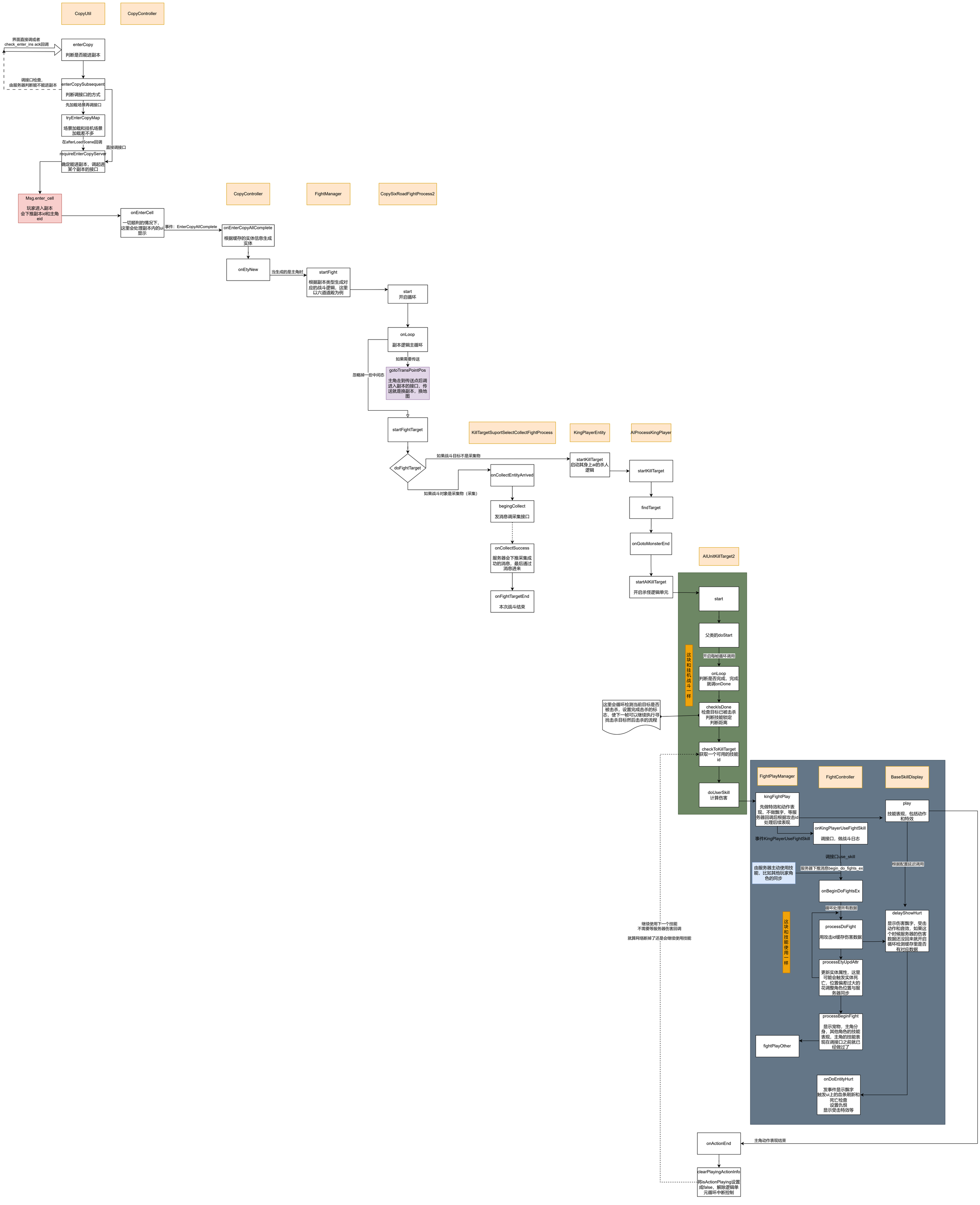


飘字



1.进入游戏后的挂机场景战斗流程


WildCopyController
MapManager
EventName.LoadDataDone
onStartWildCopy()
tryLoadMap()
加载地图
loadMap(fileName)
解析地图数据
onDataLoadDone()
加载结束后触发事件，传递fileName文件，用于后续解析
onDataDone(fileName)
地图加载完成后，通知其他模块开始初始化
onMapLoadFinish()
地图加载完成，生成地图实例，重置资源
FightManager
startFight()
启动战斗流程
Process.Run()
创建并执行战斗进程
WildFightProcess
继承自通用工具类，实现接口
start
开启线程
开始写入数据并保存战报
如果当前正在进行AI计算，则等待计算结束，保证数据一致性
开始运行
checkAllFight()
检查是否还有战斗需要处理
checkToFightTarget
查找战斗目标
startFightTarget
为目标生成具体战斗实体，设置一组战斗数据
KingPlayerEntity
AProcessKingPayer
startAllFight
执行战斗逻辑
startKillTarget
tryCheckTarget
检查目标是否满足击杀条件（如血量为0、未被保护等）
checkOneDone
检查单个战斗实体是否已结束，并触发清理
AUnAllUtilsMonster
AUnAllUtilsMonster
checkOneDone
检查单个战斗实体是否已结束，并触发清理
start
父类的start函数
父类的start函数调用
setTarget
设置目标，以及攻击对象
checkOneDone
检查单个战斗实体是否已结束，并触发清理
如果当前目标因为超时、死亡或其他原因失效，则清空目标，并通知其他系统该目标已经结束
checkTotalFightTarget
统计剩余目标，决定是继续战斗还是结束
FightPlayManager
BaseSkillDisplay
doUseSkill
计算伤害
此时UnUtilsSkill会根据技能配置，生成一个具体的技能实例，并调用其Use()方法
play
播放特效动画
动画播放完毕后，会通过回调函数通知UnUtilsSkill
doUseSkillAfterAnimation
技能动画播放完毕后的回调函数，通知其他系统技能效果已经生成，可以开始结算
FightController
onUseSkillAfterAnimation
技能动画播放完毕后的回调函数，通知其他系统技能效果已经生成，可以开始结算
onTargetCompleted
技能结束后，通知Controller，本次战斗已经结束，可以进行下一个战斗目标的选择
onUnUtilsSkillAfterAnimation
技能动画播放完毕后的回调函数，通知其他系统技能效果已经生成，可以开始结算
onTargetCompleted
技能结束后，通知Controller，本次战斗已经结束，可以进行下一个战斗目标的选择
onFightTargetCompleted
通知Controller，本次战斗已经结束，可以进行下一个战斗目标的选择
EventName.FightTargetCompleted
EventName.KillTargetComplete
onFightTargetCompleted
此时所有战斗实体已经结束，通知Controller，本次战斗已经结束，可以进行下一个战斗目标的选择
onFightTargetCompleted


CopyUtil
CopyController
界面直接调或者
check_enter_ins ack回调
enterCopy
判断是否能进副本
调接口检查，
由服务器判断能不能进副本
enterCopySubsequent
判断调接口的方式
先加载场景再调接口
tryEnterCopyMap
场景加载和挂机场景加载差不多
在afterLoadScene回调
直接调接口
requireEnterCopyServer
确定能进副本，调起进某个副本的接口
Msg.enter_cell
玩家进入副本
会下推副本id和主角eid
onEnterCell
一切顺利的情况下，这里会处理副本内的ui显示
事件：EnterCopyAllComplete
CopyController
onEnterCopyAllComplete
根据缓存的实体信息生成实体
onEtyNew
当生成的是主角时
FightManager
startFight
根据副本类型生成对应的战斗逻辑，这里以六道道殿为例
CopySixRoadFightProcess2
start
开启循环
onLoop
副本逻辑主循环
如果需要传送
gotoTransPointPos
主角走到传送点后调进入副本的接口，传送就是换副本，换地图
忽略掉一些中间态
startFightTarget
doFightTarget
如果战斗目标不是采集物
如果战斗对象是采集物（采集）
KillTargetSuportSelectCollectFightProcess
onCollectEntityArrived
begingCollect
发消息调采集接口
onCollectSuccess
服务器会下推采集成功的消息，最后通过消息进来
onFightTargetEnd
本次战斗结束
KingPlayerEntity
startKillTarget
启动其身上ai的杀人逻辑
AIProcessKingPlayer
startKillTarget
findTarget
onGotoMonsterEnd
startAIKillTarget
开启杀怪逻辑单元
AIUnitKillTarget2
start
父类的doStart
开启每帧循环调用
onLoop
判断是否完成，完成就调onDone
checkIsDone
检查目标已被击杀
判断技能锁定
判断距离
这块和挂机战斗一样
这里会循环检测当前目标是否被击杀，设置完成击杀的标志，使下一帧可以继续执行寻找击杀目标然后击杀的流程
checkToKillTarget
获取一个可用的技能id
doUserSkill
计算伤害
继续使用下一个技能
不需要等服务器伤害回调
就算网络断掉了还是会继续使用技能
FightPlayManager
FightController
BaseSkillDisplay
kingFightPlay
先做特效和动作表现，不做飘字，等服务器回调后根据攻击id处理后续表现
play
技能表现，包括动作和特效
onKingPlayerUseFightSkill
调接口，做战斗日志
事件KingPlayerUseFightSkill
调接口use_skill
由服务器主动使用技能，比如其他玩家角色的同步
服务器下推消息begin_do_fights_ex
根据配置延迟调用
onBeginDoFightsEx
循环处理所有数据
这块和技能使用一样
processDoFight
用攻击id缓存伤害数据
delayShowHurt
显示伤害飘字，受击动作和音效，如果这个时候服务器的伤害数据还没回来就开启循环检测缓存里是否有对应数据
processEtyUpdAttr
更新实体属性，这里可能会触发实体死亡，位置偏差过大的花调整角色位置与服务器同步
processBeginFight
显示宠物，主角分身，其他角色的技能表现，主角的技能表现在调接口之前就已经做过了
fightPlayOther
onDoEntityHurt
发事件显示飘字
触发ui上的血条刷新和死亡检查
设置仇恨
显示受击特效等
onActionEnd
主角动作表现结束
clearPlayingActionInfo
将isActionPlaying设置成false，解除逻辑单元循环中断控制